Cahier des Charges

Application Mobile

Bãng Zĩiga

« Découvre le pays »

Version MVP (Minimum Viable Product)

Projet de Développement Mobile Flutter
Formation Niveau Approfondi

19 janvier 2026

Table des matières

1 Présentation du projet 2

2 Objectifs du projet 2

3 Public cible 3

4 Périmètre du MVP 3

5 Description détaillée des fonctionnalités du MVP 4

6 Contraintes techniques 8

7 Architecture fonctionnelle 10

8 Parcours utilisateur (User Flow) 12

1 Présentation du projet

1.1 Contexte

Dans un contexte où le tourisme local gagne en importance et où les voyageurs recherchent des expériences authentiques, il devient essentiel de valoriser le patrimoine culturel et touristique régional. Ce projet s'inscrit dans une démarche de promotion du territoire à travers une solution numérique accessible.

1.2 Description générale

L'application mobile **Bãng Zĩiga** (« Découvre le pays ») est une plateforme numérique destinée à promouvoir les attractions touristiques et la richesse culturelle d'une région. Elle permet aux utilisateurs de découvrir les sites touristiques locaux, de s'informer sur la culture régionale et d'explorer des itinéraires de visite suggérés.

Cette application s'adresse aussi bien aux touristes visitant la région qu'aux habitants souhaitant mieux connaître leur territoire. Elle intègre un système d'authentification permettant aux utilisateurs de créer un compte personnel et de se connecter à l'application.

2 Objectifs du projet

2.1 Objectif principal

Créer une application mobile fonctionnelle permettant de valoriser le tourisme et la culture locale à travers une interface intuitive et attrayante, avec un système d'authentification utilisateur sécurisé.

2.2 Objectifs spécifiques

- Permettre aux utilisateurs de créer un compte et de s'authentifier
- Fournir un guide numérique des sites touristiques de la région
- Informer sur la culture, les traditions et la gastronomie locales
- Proposer des itinéraires de découverte prêts à l'emploi
- Offrir une expérience utilisateur fluide et agréable
- Démontrer la maîtrise des compétences en développement mobile Flutter

2.3 Objectifs techniques

- Développer une application Flutter multiplateforme (Android/iOS)
- Implémenter un système d'authentification complet
- Intégrer une communication avec une API REST
- Implémenter une gestion d'état efficace
- Assurer une architecture propre et maintenable
- Gérer correctement les états de chargement et les erreurs

3 Public cible

3.1 Utilisateurs principaux

— **Touristes** : visiteurs extérieurs souhaitant découvrir la région

— **Habitants locaux** : résidents désireux de mieux connaître leur territoire

— **Étudiants** : personnes en séjour temporaire cherchant à explorer la région

3.2 Profil des utilisateurs

— Âge : 18-65 ans

— Usage : consultation ponctuelle avant ou pendant une visite

— Niveau technologique : utilisateurs moyens de smartphones

— Besoin : information rapide, claire et fiable avec compte personnel

4 Périmètre du MVP

4.1 Fonctionnalités INCLUSES dans le MVP

4.1.1 Authentification utilisateur

— Page d'inscription (Sign Up) avec :
  — Nom complet
  — Adresse email
  — Mot de passe (avec confirmation)
  — Validation des champs

— Page de connexion (Login) avec :
  — Email
  — Mot de passe
  — Option « Se souvenir de moi »
  — Lien vers l'inscription

— Gestion de session utilisateur

— Déconnexion

— Messages d'erreur appropriés (email invalide, mot de passe faible, etc.)

4.1.2 Écran d'accueil

— Présentation de l'application

— Mise en avant de 3-4 sites phares

— Navigation vers les sections principales

— Menu utilisateur (profil, déconnexion)

4.1.3 Guide des sites touristiques

- Liste complète des sites (affichage en grille ou liste)
- Fiche détaillée de chaque site avec :
  - Nom du site
  - Description complète
  - Image représentative
  - Catégorie (monument, musée, parc, etc.)
  - Localisation (adresse textuelle)

4.1.4 Culture locale

- Page d'information sur la culture régionale
- Contenu sur les traditions locales
- Informations sur la gastronomie
- Événements culturels (liste simple)

4.1.5 Itinéraires suggérés

- Liste d'itinéraires touristiques prédéfinis
- Fiche détaillée d'un itinéraire avec :
  - Nom de l'itinéraire
  - Description du parcours
  - Durée estimée
  - Sites inclus dans l'itinéraire
  - Niveau de difficulté (facile, moyen, intense)

4.1.6 Aspects techniques

- Navigation fluide entre les écrans
- Gestion des états : chargement, succès, erreur
- Messages d'erreur explicites
- Interface responsive
- Persistance de la session utilisateur

5 Description détaillée des fonctionnalités du MVP

5.1 Système d'authentification

5.1.1 Page d'inscription

Objectif : Permettre aux nouveaux utilisateurs de créer un compte
Éléments visuels :

- Logo de l'application

— Titre « Inscription » ou « Créer un compte »
— Champ « Nom complet »
— Champ « Email »
— Champ « Mot de passe » (avec icône pour afficher/masquer)
— Champ « Confirmer le mot de passe »
— Bouton « S'inscrire » (couleur principale : evergreen)
— Lien « Déjà un compte ? Se connecter »
— Messages de validation sous chaque champ

Comportement :

— Validation en temps réel des champs
— Vérification de la force du mot de passe
— Vérification de la correspondance des mots de passe
— Validation du format email
— Envoi des données à l'API d'inscription
— Affichage d'un indicateur de chargement
— Redirection vers la page de connexion après inscription réussie
— Messages d'erreur si email déjà utilisé ou autres problèmes

5.1.2 Page de connexion

Objectif : Permettre aux utilisateurs existants de se connecter
Éléments visuels :

— Logo de l'application
— Titre « Connexion » ou « Bienvenue »
— Champ « Email »
— Champ « Mot de passe » (avec icône pour afficher/masquer)
— Case à cocher « Se souvenir de moi »
— Bouton « Se connecter » (couleur principale : evergreen)
— Lien « Pas encore de compte ? S'inscrire »
— Messages d'erreur sous les champs si nécessaire

Comportement :

— Validation des champs obligatoires
— Envoi des identifiants à l'API
— Affichage d'un indicateur de chargement
— Stockage du token d'authentification si connexion réussie
— Redirection vers l'écran d'accueil
— Messages d'erreur appropriés (identifiants incorrects, problème réseau)
— Mémorisation de la session si option activée

5.2 Écran d'accueil

Objectif : Accueillir l'utilisateur connecté et lui présenter l'application
Éléments visuels :

- Barre de navigation avec :
  - Logo/titre de l'application
  - Icône de profil utilisateur
  - Icône de menu
- Message de bienvenue personnalisé (« Bonjour [Nom] »)
- Section « Sites phares » avec 3-4 cartes visuelles
- Boutons d'accès rapide aux sections principales :
  - Guide des sites (couleur sage)
  - Culture locale (couleur moss)
  - Itinéraires (couleur evergreen)

Comportement :

- Vérification de l'authentification au chargement
- Chargement automatique des sites phares depuis l'API
- Affichage d'un indicateur de chargement
- Gestion des erreurs de connexion avec message approprié
- Menu déroulant avec option de déconnexion

5.3 Guide des sites touristiques

5.3.1 Liste des sites

Objectif : Afficher tous les sites touristiques disponibles
Éléments visuels :

- Grille ou liste de cartes représentant chaque site
- Chaque carte contient :
  - Image du site
  - Nom du site
  - Catégorie (badge coloré avec couleur sage)
  - Indicateur de localisation
- Barre de navigation en haut

Comportement :

- Récupération de la liste depuis l'API REST
- Affichage progressif des éléments
- Gestion de l'état vide (aucun site disponible)
- Navigation vers la fiche détaillée au clic sur une carte
- Animation de transition fluide

5.3.2 Fiche détaillée d'un site

Objectif : Fournir des informations complètes sur un site spécifique
Éléments visuels :

- Image principale en plein écran (bandeau supérieur)
- Bouton retour (couleur evergreen)
- Nom du site (titre en couleur moss)
- Catégorie et localisation (badges avec couleurs de la palette)
- Description complète (texte scrollable)
- Icônes informatives

Comportement :

- Chargement des détails depuis l'API
- Possibilité de faire défiler le contenu
- Animation de transition hero pour l'image
- Gestion des erreurs de chargement

5.4 Culture locale

Objectif : Informer sur la richesse culturelle de la région
Éléments visuels :

- En-tête avec image représentative
- Sections organisées avec codes couleur :
  - Traditions locales (sage)
  - Gastronomie (moss)
  - Événements culturels (evergreen)
- Texte formaté avec images d'illustration
- Navigation par onglets ou sections scrollables

Comportement :

- Chargement du contenu depuis l'API ou données statiques
- Navigation fluide entre les sections
- Affichage adaptatif selon la longueur du contenu

5.5 Itinéraires suggérés

5.5.1 Liste des itinéraires

Objectif : Proposer des parcours de visite prédéfinis
Éléments visuels :

- Liste de cartes d'itinéraires
- Chaque carte contient :
  - Nom de l'itinéraire

  - Durée estimée (icône horloge en moss)
  - Niveau de difficulté (icône avec couleur appropriée)
  - Image représentative
  - Nombre de sites inclus (badge en evergreen)

Comportement :

- Récupération depuis l'API
- Navigation vers le détail au clic
- Animation de transition

5.5.2 Détail d'un itinéraire

Objectif : Présenter le parcours complet
Éléments visuels :

- Image de couverture
- Nom de l'itinéraire (titre en moss)
- Informations clés avec icônes colorées :
  - Durée (sage)
  - Niveau (moss)
  - Distance approximative (evergreen)
- Description du parcours
- Liste des sites inclus (avec mini-cartes cliquables)

Comportement :

- Chargement des détails depuis l'API
- Possibilité de cliquer sur un site pour voir sa fiche
- Scrolling fluide
- Navigation inter-écrans

6 Contraintes techniques

6.1 Technologies obligatoires

Framework : Flutter (version stable récente)
Langage : Dart
Architecture :

- Structure modulaire (dossiers séparés pour models, services, screens, widgets)
- Séparation des responsabilités
- Pattern Repository pour la gestion des données

Gestion d'état :

- Provider, Riverpod, ou BLoC (au choix selon préférence)
- Gestion claire des états : initial, loading, success, error

— État d'authentification global

Communication API :

— Package `http` ou `dio` pour les requêtes REST

— Gestion des erreurs réseau

— Timeouts configurés

— Intercepteurs pour le token d'authentification

Navigation :

— Utilisation du système de navigation Flutter natif

— Routes nommées recommandées pour la clarté

— Guard de navigation pour protéger les routes authentifiées

Stockage local :

— `shared_preferences` pour le token et les préférences

— Stockage sécurisé du token d'authentification

6.2 Packages recommandés

— `http` ou `dio` : requêtes API

— `provider` ou `riverpod` : gestion d'état

— `shared_preferences` : stockage local

— `cached_network_image` : mise en cache des images

— `google_fonts` : typographie personnalisée

— `flutter_launcher_icons` : génération d'icônes

— `flutter_svg` : support des images SVG

— `email_validator` : validation des emails

6.3 Palette de couleurs Sage Green

L'application utilisera la palette de couleurs monochromatique Sage Green :

— **Sage Hint** : `#BFCFBB` - Arrière-plans, éléments secondaires

— **Mint** : `#BFCFBB` - Cartes, conteneurs

— **Sage** : `#8EA58C` - Éléments interactifs, badges

— **Moss** : `#738A6E` - Titres, textes importants

— **Evergreen** : `#344C3D` - Boutons principaux, navigation

Cette palette devra être définie dans un fichier de constantes et utilisée de manière cohérente dans toute l'application.

6.4 API REST

Source de données :

— API publique de tourisme (si disponible)

— API mock créée avec Mocky.io, JSONPlaceholder, ou serveur local simple

— JSON statique hébergé (solution de secours)

— Firebase (optionnel pour l'authentification)

Endpoints nécessaires :

```
Authentification :
- POST /auth/register : inscription
- POST /auth/login : connexion
- POST /auth/logout : déconnexion

Données :
- GET /sites : liste des sites touristiques
- GET /sites/{id} : détail d'un site
- GET /culture : informations culturelles
- GET /itineraires : liste des itinéraires
- GET /itineraires/{id} : détail d'un itinéraire
```

Format des réponses : JSON
Authentification : Bearer Token (JWT recommandé)

6.5 Contraintes de performance

— Temps de chargement initial ¡ 3 secondes

— Fluidité de navigation (60 fps)

— Gestion efficace de la mémoire (images optimisées)

— Pas de blocage de l'interface utilisateur

— Validation rapide de l'authentification

7 Architecture fonctionnelle

7.1 Vue d'ensemble de l'architecture

L'application suivra une architecture en couches pour assurer la maintenabilité et la scalabilité :

```
        INTERFACE (UI)
  Screens / Pages / Widgets



     GESTION D'ÉTAT (State)
```

```
 Providers / BLoC / Controllers


       SERVICES (Business)
 API Service, Auth Service


     REPOSITORIES (Data)
 Auth Repository, Sites Repository


        MODELS (Data)
 User, Site, Itineraire, Culture
```

7.2 Organisation des dossiers

```
lib/
 main.dart
 models/
    user.dart
    site.dart
    itineraire.dart
    culture_info.dart
 services/
    api_service.dart
    auth_service.dart
    storage_service.dart
 repositories/
    auth_repository.dart
    sites_repository.dart
    itineraires_repository.dart
 providers/
    auth_provider.dart
    sites_provider.dart
    itineraires_provider.dart
    culture_provider.dart
 screens/
    auth/
       login_screen.dart
       signup_screen.dart
    home_screen.dart
    sites_list_screen.dart
    site_detail_screen.dart
    culture_screen.dart
```

```
    itineraires_list_screen.dart
    itineraire_detail_screen.dart
 widgets/
    site_card.dart
    itineraire_card.dart
    custom_button.dart
    custom_text_field.dart
    loading_widget.dart
    error_widget.dart
 utils/
     colors.dart
     text_styles.dart
     constants.dart
     validators.dart
```

7.3 Flux de données

7.3.1 Flux d'authentification

1. **Inscription** : L'utilisateur remplit le formulaire
2. **Validation** : Les données sont validées côté client
3. **Requête API** : Envoi des données au serveur
4. **Réponse** : Confirmation ou erreur
5. **Redirection** : Vers la page de connexion si succès
6. **Connexion** : L'utilisateur saisit ses identifiants
7. **Authentification** : Vérification par l'API
8. **Token** : Stockage du token d'authentification
9. **État global** : Mise à jour de l'état d'authentification
10. **Navigation** : Redirection vers l'écran d'accueil

7.3.2 Flux de données principal

1. **Initialisation** : Vérification de la session au lancement
2. **Requête API** : Le service API récupère les données avec le token
3. **Mise à jour de l'état** : Le provider notifie les changements
4. **Affichage** : L'interface se met à jour automatiquement
5. **Interaction** : L'utilisateur navigue et consulte les détails
6. **Gestion des erreurs** : Messages appropriés en cas de problème

8 Parcours utilisateur (User Flow)

8.1 Scénario principal : Premier lancement et inscription

1. Lancement de l'application

2. Affichage de la page de connexion
3. Clic sur « Pas encore de compte ? S'inscrire »
4. Remplissage du formulaire d'inscription
5. Validation et création du compte
6. Redirection vers la page de connexion
7. Connexion avec les nouveaux identifiants
8. Arrivée sur l'écran d'accueil personnalisé

8.2 Scénario secondaire : Découverte d'un site touristique

1. Connexion réussie
2. Affichage de l'écran d'accueil
3. Visualisation des sites phares
4. Navigation vers « Guide des sites »
5. Consultation de la liste complète
6. Scroll pour explorer les options
7. Sélection d'un site intéressant
8. Lecture de la fiche détaillée
9. Retour à la liste ou navigation vers un autre site

8.3 Scénario tertiaire : Consultation d'un itinéraire

1. Depuis l'accueil ou le menu
2. Navigation vers « Itinéraires »
3. Consultation de la liste des parcours
4. Sélection d'un itinéraire adapté
5. Lecture du détail du parcours
6. Consultation des sites inclus
7. Possibilité de cliquer sur un site pour voir sa fiche

8.4 Scénario quaternaire : Déconnexion

1. Depuis n'importe quel écran
2. Ouverture du menu utilisateur
3. Clic sur « Déconnexion »
4. Confirmation de la déconnexion
5. Suppression du token
6. Redirection vers la page de connexion